Quang Tran

(508) 633-2195 | QTran1018@gmail.com | quangntran.com | linkedin.com/in/qtran1018 | github.com/qtran1018

SUMMARY

Built two full-stack applications as lead developer: a 47,000+ record bilingual card recognition system with sub-second scan latency, and a four-app travel platform covering microservices, SSO, AI integration, and containerized deployment. B.S. Computer Science, Feb 2025. Prior background in SQL pipeline automation at a SaaS company.

TECHNICAL SKILLS

Languages: Java, Python, JavaScript, TypeScript, SQL, HTML5, CSS3, C++
Frameworks & Libraries: Spring Boot, FastAPI, Django, Flask, Express, React.js, Vue.js, Node.js, Pandas, NumPy, scikit-learn
Databases & Caching: PostgreSQL, MySQL, MongoDB, Redis, pgvector, SQLite
Cloud & Infrastructure: Docker, CI/CD (GitHub Actions, Jenkins), AWS (EC2, S3, RDS) (coursework)
APIs & Auth: REST APIs, JWT, Google OAuth 2.0, Keycloak OIDC, Django REST Framework
Testing & Dev Tools: Playwright (E2E), Postman, Git, GitHub, Jira, Figma, Agile/Scrum, SDLC

PROJECTS

TCG Card Scanner | *React Native (Expo) · FastAPI · PostgreSQL + pgvector · Redis · YOLO · CLIP · Docker*

- Cut card identification latency to 700-900 ms by collapsing a two-step detect-then-recognize flow into a single backend roundtrip combining on-device YOLO cropping and CLIP similarity search.
- Improved match precision across a 47,000+ card bilingual catalog by building a multi-signal PostgreSQL ranking pipeline: pg_trgm fuzzy name match, card-number spatial filter, and a first-word penalty that prevents weaker matches from outranking exact ones.
- Prevented cross-game false positives in a shared pgvector index by scoping embedding searches to a game column and adding a consecutive-frame confirmation gate before committing live-scan results.
- Caught streaming contract regressions before deploy by writing Playwright end-to-end tests against the NDJSON price endpoint across 12 card variants, integrated into GitHub Actions CI as a required check.

TravelBin | *Django · React · PostgreSQL · Keycloak · Docker | Collaborative (lead)*

- Owned the full backend architecture of a collaborative travel planning app, delivering Django ORM models, relational schema, DRF serializers, and the complete test suite before any collaborator wrote a line, then enforced code quality as lead reviewer on all subsequent GitHub PRs.
- Replaced a naive ownership check with a join-table permission model (user ID x destination ID), designed with a senior engineer collaborator, enabling multi-user edit access per trip; covered with 15+ pytest cases across auth, 404, and denial scenarios.
- Delivered a responsive frontend where trip entries auto-save without user action by building a React 19 SPA with 600 ms debounced PATCH calls, Keycloak-js SSO, and TanStack Query cache invalidation on every write.
- Reduced invite-to-member friction to zero clicks post-login by implementing sessionStorage-backed auto-join on OAuth return, handling the full unauthenticated-to-member path without a manual join step.

Intonational + Itinerary-Agent | *FastAPI · Vue 3 · Express · MongoDB · Redis · OpenAI · Docker · Keycloak*

- Integrated OpenAI to generate structured travel itineraries by designing a constrained JSON prompt schema, validating output server-side in Express, persisting it via Prisma, and proxying entries to TravelBin for one-click import across apps.
- Eliminated cache-layer 500 errors under quota pressure by implementing Redis TTL caching with lazy hydration and graceful degradation across a three-service FastAPI backend serving weather and currency rate data.
- Unified SSO across all four platform apps by configuring a Keycloak OIDC realm with per-client PKCE and confidential flows, reducing per-app auth boilerplate to a shared JWT validation module.

Splitpush | *Spring Boot · Java · PostgreSQL · Docker · Render | splitpush.onrender.com*

- Applied object-oriented design principles to architect a layered Spring Boot application (controller, service, repository) with a fully normalized relational schema, deployed to Render and used by real users on a group trip to track shared expenses and settle balances.
- Reduced redundant database reads by applying Caffeine in-memory caching to user, group, and balance lookups, and added pagination to list endpoints to bound response size under load.

EXPERIENCE

Foundry — Needham Heights, MA
Data Report Specialist — June 2018 – April 2023

- Cut per-report processing time from ~5 minutes to under 30 seconds, then to near-instant, by self-teaching VBA to automate data verification, cleaning, and formatting across 20-25 daily lead generation reports; identified full-sheet traversal as the bottleneck by instrumenting run time, then constrained iteration to the active data range to eliminate the slowdown.
- Delivered error-free client reports by building conditional formatting into the automation to flag data integrity issues inline, replacing a manual scan and giving the team a visual, auditable anomaly log before every send.

ACADEMIC PROJECTS

WGU coursework demonstrating object-oriented design, relational modeling, and algorithm implementation in Java and Python.

- Inventory Management System (Java, JavaFX): implemented OOP composition pattern with UML-designed class hierarchy, part-product relational integrity, and input validation across a full CRUD desktop application.
- Appointment Scheduler (Java, JDBC, MySQL): built an ERD-driven scheduling app with role-based login, time zone and locale detection for US/UK/Canada/French users, and business hours enforcement via date-time validation logic.
- Package Delivery Router (Python): implemented a custom hash map for O(1) package lookups and a nearest-neighbor greedy algorithm to optimize multi-truck delivery routes across a distance matrix, minimizing total mileage under time constraints.

EDUCATION

Western Governors University — Online
B.S. in Computer Science — Completed February 2025